

LAB3: Computer Vision

Feature Matching Strategies — Brute-Force vs FLANN + Ratio Test

Objective:

- To implement and compare Brute-Force and FLANN-based feature matching strategies and apply Lowe's ratio test to improve matching accuracy using OpenCV in Python.

LAB: Computer Vision

Description

Feature matching is the process of finding similar keypoints between two images.

LAB: Computer Vision

Image 1 → Detect Keypoints → Extract Descriptors

Image 2 → Detect Keypoints → Extract Descriptors



Matching Algorithm



Best Corresponding Points

Keypoints

Important points
in an image such
as:

Corners

Edges

Blobs

LAB: Computer Vision

Descriptors

Numerical representation (feature vector) of keypoints.

Example:

ORB descriptor = Binary vector

SIFT descriptor = 128-dimension vector

LAB: Computer Vision

Matching compares descriptors from:

Image 1 (Query)

Image 2 (Train)

- Distance Metrics:

Hamming Distance (ORB)

Euclidean Distance (SIFT)

LAB: Computer Vision

Brute-Force Matcher

Brute-Force matcher compares **each descriptor of image A with all descriptors of image B.**

$$d = \min(\| D_1 - D_2 \|)$$

It selects the closest match based on distance.

LAB: Computer Vision

FLANN Matcher

FLANN = Fast Library for Approximate Nearest Neighbors It uses intelligent search structures (KD-Tree / LSH).

*Instead of checking ALL descriptors (like BF),
FLANN finds approximate nearest matches quickly.*

LAB: Computer Vision

Ratio Test

- Ratio test removes false matches by comparing: Best match (m) Second best match (n).

$$\frac{m.distance}{n.distance} < 0.75$$

Small ratio → Good match

Large ratio → False match


```
[1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
[2]: img1 = cv2.imread('Img1.jpg', 0) # Query Image (Grayscale)
img2 = cv2.imread('Img2.jpg', 0) # Train Image
```

```
plt.imshow(img1, cmap='gray')
plt.title("Image 1")
plt.axis('off')
plt.show()
```

```
plt.imshow(img2, cmap='gray')
plt.title("Image 2")
plt.axis('off')
plt.show()
```

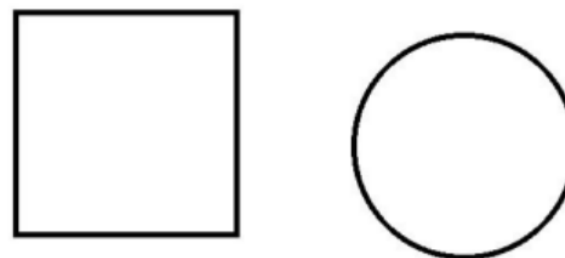
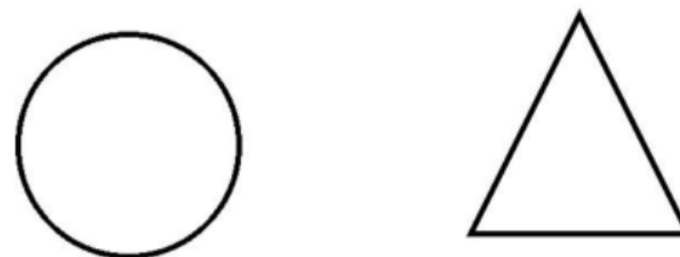


Image 2

Feature Matching Demo Scene



```
[3]: # Create ORB detector
orb = cv2.ORB_create(nfeatures=500)

# Detect and compute
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

print("Keypoints in Image 1:", len(kp1))
print("Keypoints in Image 2:", len(kp2))
```

Keypoints in Image 1: 376

Keypoints in Image 2: 377

```
[4]: # Create BF Matcher with Hamming distance (for ORB)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

# Match descriptors
matches = bf.match(des1, des2)

# Sort matches by distance (best first)
matches = sorted(matches, key=lambda x: x.distance)

print("Total BF Matches:", len(matches))
```

Total BF Matches: 147

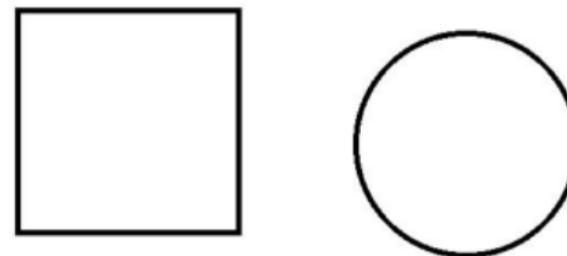
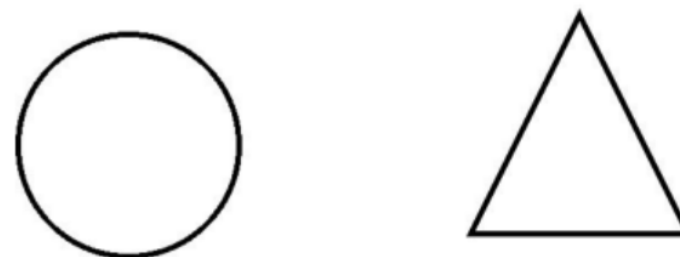


Image 2

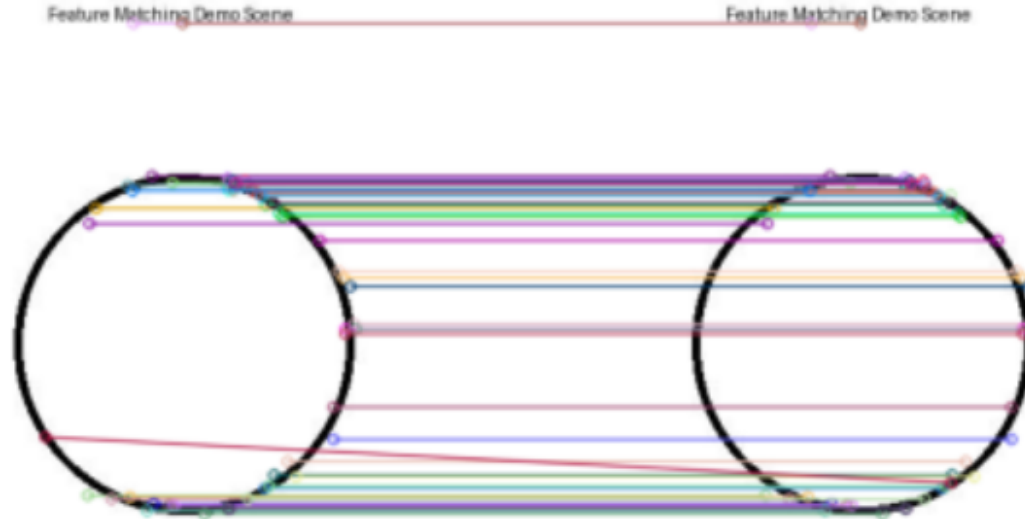
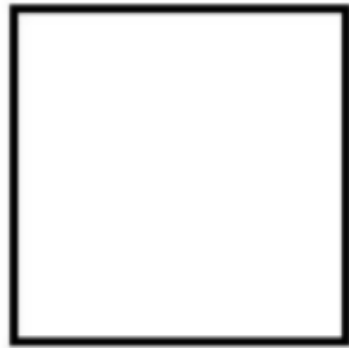
Feature Matching Demo Scene



```
[5]: bf_result = cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None, flags=2)

plt.figure(figsize=(12,6))
plt.imshow(bf_result, cmap='gray')
plt.title("Brute-Force Matching Results")
plt.axis('off')
plt.show()
```

Brute-Force Matching Results



```
[6]: FLANN_INDEX_LSH = 6

index_params = dict(algorithm=FLANN_INDEX_LSH,
                    table_number=6,
                    key_size=12,
                    multi_probe_level=1)

search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)
```

```
[7]: matches = flann.knnMatch(des1, des2, k=2)

print("Total KNN Matches:", len(matches))
```

Total KNN Matches: 376

```
[6]: FLANN_INDEX_LSH = 6

index_params = dict(algorithm=FLANN_INDEX_LSH,
                    table_number=6,
                    key_size=12,
                    multi_probe_level=1)

search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)
```

```
[7]: matches = flann.knnMatch(des1, des2, k=2)

print("Total KNN Matches:", len(matches))
```

Total KNN Matches: 376

```
[8]: good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

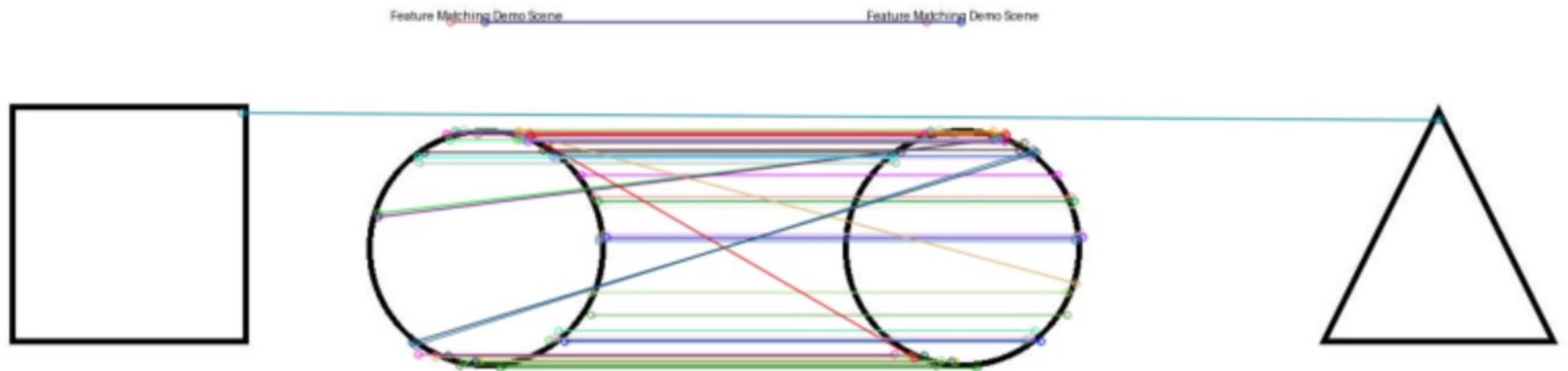
print("Good Matches after Ratio Test:", len(good_matches))
```

Good Matches after Ratio Test: 142

```
[9]: flann_result = cv2.drawMatches(img1, kp1, img2, kp2, good_matches[:50], None, flags=2)

plt.figure(figsize=(12,6))
plt.imshow(flann_result, cmap='gray')
plt.title("FLANN + Ratio Test Matching")
plt.axis('off')
plt.show()
```

FLANN + Ratio Test Matching



-Students task:

- Task 1: Implement ORB + BF Matcher and show top 30 matches.
- Task 2: Implement FLANN + Ratio Test and compare results.
- Task 3 (Advanced): Change ratio threshold:

- *At the end create pdf file and submit, don't be late, late submission may affect your sessional marks*